

NiceGUI 中 app.native 的详细解析

`app.native` 是 NiceGUI 暴露底层原生组件 / 实例的核心属性，其本质是对 NiceGUI 封装的底层框架（FastAPI、Quasar、Uvicorn 等）的直接访问入口。通过 `app.native`，开发者可以突破 NiceGUI 的高层封装，直接操作底层框架的核心功能，实现定制化、高性能的扩展开发。以下从核心结构、关键属性、使用场景、进阶示例等维度全面解析：

一、核心定位与底层依赖

NiceGUI 的底层技术栈：

- 后端：基于 FastAPI（ASGI Web 框架）+ Uvicorn（ASGI 服务器）；
- 前端：基于 Quasar（Vue.js 组件库）+ WebSocket（前后端实时通信）；
- `app.native` 的核心作用：将这些底层框架的实例 / API 暴露给开发者，实现「高层封装不够用，直接操作底层」的需求。

`app.native` 的结构（简化版）：

```
app.native = {
  'app': FastAPI实例,           # 核心: FastAPI应用实例
  'uvicorn_config': Uvicorn配置对象, # Uvicorn服务器配置
  'websocket_manager': 内置WebSocket管理器, # 前后端实时通信核心
  'quasar_config': Quasar前端配置, # 前端Quasar框架配置
}
```

二、核心属性详解

1. `app.native.app`：FastAPI 实例（最常用）

`app.native.app` 是 NiceGUI 底层的 FastAPI 应用实例，完全兼容 FastAPI 的所有 API，是 `app.native` 中最核心、最常用的属性。

核心能力：

- 注册 FastAPI 原生路由（补充 NiceGUI 的 `app.urls`）；
- 添加 FastAPI 中间件（如 CORS、认证、日志）；
- 配置 FastAPI 的依赖、响应模型、文档（Swagger/ReDoc）；
- 访问 FastAPI 的请求 / 响应生命周期、状态管理。

示例 1：注册 FastAPI 原生路由

（补充 NiceGUI 路由的不足，如高性能 API、OpenAPI 文档自动生成）

```
from nicegui import app, ui
from pydantic import BaseModel
```

```
# 定义FastAPI响应模型（自动生成OpenAPI文档）
class UserResponse(BaseModel):
    id: int
    name: str
    email: str

# 通过app.native.app直接注册FastAPI路由
@app.native.app.get('/api/fastapi/user/{user_id}', response_model=UserResponse)
def fastapi_user_api(user_id: int):
    # 原生FastAPI路由，支持自动参数校验、响应模型、Swagger文档
    return {'id': user_id, 'name': '张三', 'email': f'zhangsan{user_id}@example.com'}

ui.label('访问 http://localhost:8080/docs 查看FastAPI自动生成的API文档')
ui.run()
```

- 访问 `http://localhost:8080/docs` 可看到 FastAPI 自动生成的 Swagger 文档（NiceGUI 的 `app.urls` 注册的路由不会出现在该文档中）；
- 优势：FastAPI 原生路由支持参数校验、响应模型、依赖注入等高级特性，适合高性能 API 开发。

示例 2：添加 FastAPI 中间件

(如全局日志、CORS 跨域、身份验证)

```
from fastapi import Request
from fastapi.middleware.cors import CORSMiddleware
import time

# 1. 添加CORS中间件（解决跨域）
app.native.app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'], # 生产环境指定具体域名
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

# 2. 添加自定义日志中间件
@app.native.app.middleware('http')
async def log_middleware(request: Request, call_next):
    start_time = time.time()
    # 执行请求处理
    response = await call_next(request)
    # 记录日志
    process_time = time.time() - start_time
    response.headers['X-Process-Time'] = str(process_time)
    print(f'请求路径: {request.url.path}, 耗时: {process_time:.2f}秒')
    return response

ui.label('已添加FastAPI全局中间件')
ui.run()
```

2. `app.native.uvicorn_config`: Uvicorn 配置对象

`app.native.uvicorn_config` 是 NiceGUI 启动时创建的 Uvicorn 服务器配置实例，包含服务器的所有运行参数（端口、主机、工作进程、日志级别等）。

核心能力：

- 动态修改 Uvicorn 配置（如运行时调整日志级别）；
- 查看当前服务器的运行状态（如绑定的地址、工作进程数）。

示例：查看 / 修改 Uvicorn 配置

```
from nicegui import app, ui

# 启动前查看默认Uvicorn配置
print('默认Uvicorn配置: ', app.native.uvicorn_config)

# 自定义Uvicorn配置并启动
ui.run(
    host='0.0.0.0',
    port=9000,
    uvicorn_logging_level='debug',
    workers=2
)

# 启动后查看修改后的配置
print('启动后Uvicorn配置: ', app.native.uvicorn_config)
# 输出示例: UvicornConfig(app=<fastapi.applications.FastAPI object>, host='0.0.0.0',
port=9000, ...)
```

3. `app.native.websocket_manager`: WebSocket 管理器

NiceGUI 的前后端实时通信（如组件状态同步、事件触发）依赖内置的 WebSocket 管理器，`app.native.websocket_manager` 是该管理器的实例，负责管理所有客户端的 WebSocket 连接。

核心能力：

- 主动向指定客户端推送消息；
- 广播消息到所有连接的客户端；
- 查看当前连接的客户端数量 / ID。

示例：主动推送 WebSocket 消息

```
from nicegui import app, ui
import asyncio

# 向所有客户端广播消息
async def broadcast_message():
    while True:
        # 通过websocket_manager主动推送消息
        await app.native.websocket_manager.broadcast({
```

```

        'type': 'custom_message',
        'content': f'当前时间: {asyncio.get_event_loop().time():.2f}'
    })
    await asyncio.sleep(1)

# 前端接收WebSocket消息并显示
ui.add_head_html('''
<script>
    // 监听NiceGUI的WebSocket消息
    window.socket.addEventListener('message', (event) => {
        const data = JSON.parse(event.data);
        if (data.type === 'custom_message') {
            document.getElementById('message').innerText = data.content;
        }
    });
</script>
''')
ui.label('实时消息: ').id('message')

# 启动后台任务推送消息
ui.run(on_startup=lambda: asyncio.create_task(broadcast_message()))

```

- 原理：NiceGUI 的前端通过 WebSocket 与后端保持连接，`websocket_manager` 负责管理这些连接并推送消息；
- 场景：实时监控、消息通知、多人协作等功能。

4. `app.native.quasar_config`：Quasar 前端配置

`app.native.quasar_config` 用于配置前端 Quasar 框架的全局属性（如主题、图标、语言、全局组件）。

示例：修改 Quasar 全局主题

```

from nicegui import app, ui

# 配置Quasar主题为暗色模式
app.native.quasar_config['dark'] = True
# 配置Quasar默认语言为中文
app.native.quasar_config['lang'] = 'zh-CN'

ui.label('Quasar暗色模式 + 中文语言')
ui.button('切换主题', on_click=lambda: ui.dark_mode.toggle())
ui.run()

```

- 支持的配置项：参考 Quasar 官方文档（如 `theme`、`iconSet`、`rtl` 等）；
- 注意：需在 `ui.run()` 前配置，否则可能不生效。

三、关键使用场景

1. 扩展 NiceGUI 的 Web 能力

- 当 NiceGUI 的 `app.urls` 满足不了复杂 API 需求时（如 OpenAPI 文档、参数校验），使用 `app.native.app` 注册 FastAPI 原生路由；
- 需全局中间件（如 CORS、日志、认证）时，通过 `app.native.app` 添加 FastAPI 中间件。

2. 实时通信与消息推送

- 基于 `app.native.websocket_manager` 实现主动推送（如实时数据更新、告警通知），替代 NiceGUI 的 `ui.notify`（仅前端提示）。

3. 定制化服务器配置

- 通过 `app.native.uvicorn_config` 动态调整 Uvicorn 参数（如运行时修改日志级别、工作进程数）。

4. 集成第三方 FastAPI 扩展

- FastAPI 生态的所有扩展（如 `fastapi-jwt-auth`、`fastapi-cache2`、`fastapi-sqlalchemy`）都可通过 `app.native.app` 集成到 NiceGUI 中：

```
from fastapi_jwt_auth import AuthJWT
from pydantic import BaseModel

class Settings(BaseModel):
    authjwt_secret_key: str = "secret"

@AuthJWT.load_config
def get_config():
    return Settings()

# 将JWT扩展绑定到NiceGUI的FastAPI实例
AuthJWT(app.native.app)

@app.native.app.post('/api/login')
def login(Authorize: AuthJWT = Depends()):
    access_token = Authorize.create_access_token(subject="user123")
    return {"access_token": access_token}
```

四、注意事项

1. 版本兼容性

- `app.native` 的属性依赖 NiceGUI 的底层实现，不同版本的 NiceGUI 可能调整 `app.native` 的结构（如早期版本 `app.native` 仅暴露 `app` 和 `uvicorn_config`）；
- 建议查看 NiceGUI 官方源码（`nicegui/app.py`）确认 `app.native` 的具体属性。

2. 避免与 NiceGUI 高层 API 冲突

- 若同时使用 `app.urls` 和 `app.native.app` 注册路由，需确保路径不冲突（路由匹配优先级：FastAPI 原生路由 > NiceGUI 路由）；
- 修改 `app.native.websocket_manager` 时，避免破坏 NiceGUI 的前后端通信（如修改 WebSocket 消息格式）。

3. 异步 / 同步注意事项

- FastAPI 原生路由支持异步（`async def`）和同步（`def`），而 NiceGUI 的路由处理函数默认同步；
- 操作 `app.native.websocket_manager` 时需使用异步（`await`），需在异步上下文（如 `async def` 函数、`asyncio.create_task`）中执行。

NiceGUI 中 `app.native.window_args` 的详细解析

`app.native.window_args` 是 NiceGUI 针对桌面端应用场景暴露的核心配置属性，用于定制化通过 `ui.run()` 启动的桌面窗口（基于 PyWebView 封装）的行为与外观。它仅在「桌面模式」（即 `ui.run()` 的 `native` 参数为 `True` 时）生效，是突破 NiceGUI 默认窗口样式、实现桌面应用级定制的关键入口。以下从核心定位、配置项、使用场景、示例等维度全面解析：

一、核心定位与底层依赖

1. 适用场景

NiceGUI 支持两种运行模式：

- Web 模式（默认）**：以网页形式运行（浏览器访问），`app.native.window_args` 无作用；
- 桌面模式**：通过 `ui.run(native=True)` 启动独立桌面窗口（基于 PyWebView 库，底层调用 OS 原生窗口 API），`app.native.window_args` 用于配置该窗口的所有属性。

2. 底层原理

`app.native.window_args` 本质是传递给 PyWebView `create_window()` 函数的参数字典，NiceGUI 将其封装为 `app.native` 的属性，开发者可直接修改该字典，实现对桌面窗口的精细化控制（如尺寸、标题、是否可缩放、置顶等）。

二、核心配置项详解

`app.native.window_args` 是一个字典，支持 PyWebView `create_window()` 的所有参数（不同操作系统支持的参数略有差异），核心配置项如下：

配置项	类型	默认值 (NiceGUI)	作用	跨平台支持
<code>title</code>	str	<code>'NiceGUI'</code>	窗口标题栏文本	Windows/macOS/Linux
<code>width</code>	int	<code>800</code>	窗口宽度（像素）	全平台

配置项	类型	默认值 (NiceGUI)	作用	跨平台支持
height	int	600	窗口高度（像素）	全平台
x	int	None	窗口左上角横坐标（None = 居中）	全平台
y	int	None	窗口左上角纵坐标（None = 居中）	全平台
resizable	bool	True	窗口是否可缩放	全平台
min_size	tuple	(400, 300)	窗口最小尺寸（宽，高）	全平台
max_size	tuple	None	窗口最大尺寸（宽，高）	全平台
fullscreen	bool	False	是否全屏启动	全平台
hidden	bool	False	启动时是否隐藏窗口	全平台
frameless	bool	False	是否无边框窗口（无标题栏 / 关闭按钮）	Windows/macOS
always_on_top	bool	False	窗口是否置顶（总在其他窗口上层）	全平台
transparent	bool	False	窗口是否透明（仅无边框时生效）	Windows/macOS
icon	str	None	窗口图标路径（支持.ico/.png）	Windows/macOS/Linux
zoom	float	1.0	窗口内容缩放比例	全平台

系统专属配置项

配置项	系统	作用
taskbar_icon	Windows	任务栏图标路径（区别于窗口图标）
shadow	macOS	是否显示窗口阴影（frameless 时生效）
decorations	Linux	是否显示窗口装饰（标题栏 / 边框）

三、使用方式与示例

1. 基础用法：修改窗口尺寸与标题

```
from nicegui import app, ui

# 配置桌面窗口属性（需在ui.run()前修改）
app.native.window_args.update({
    'title': '我的NiceGUI桌面应用', # 自定义标题
    'width': 1000, # 宽度1000px
    'height': 700, # 高度700px
    'resizable': False, # 禁止缩放窗口
    'always_on_top': True # 窗口置顶
})

ui.label('自定义桌面窗口示例')
# 启动桌面模式（native=True）
ui.run(native=True)
```

2. 进阶示例：无边框透明窗口

```
from nicegui import app, ui

# 配置无边框+透明窗口
app.native.window_args.update({
    'frameless': True, # 无边框（无标题栏/关闭按钮）
    'transparent': True, # 透明背景
    'width': 500,
    'height': 300,
    'always_on_top': True,
    'shadow': False # macOS: 关闭阴影（可选）
})

# 自定义UI（透明背景下的内容）
ui.label('无边框透明窗口').classes('text-2xl text-white')
ui.button('关闭窗口', on_click=lambda: app.native.window.destroy()).classes('mt-4')

# 启动桌面模式
ui.run(native=True, dark_mode=True)
```

- 注意：透明窗口仅在 `frameless=True` 时生效，且不同操作系统的透明效果略有差异（Windows 需启用硬件加速）；
- 无边框窗口需自定义关闭 / 最小化按钮（通过 `app.native.window.destroy()` 关闭窗口）。

3. 固定窗口位置 + 自定义图标

```
from nicegui import app, ui

# 配置窗口位置（屏幕左上角x=200, y=100）+ 自定义图标
app.native.window_args.update({
    'x': 200,
    'y': 100,
    'icon': './my_app_icon.ico', # 本地图标文件（Windows推荐.ico, macOS推荐.icns）
    'min_size': (600, 400),      # 最小尺寸限制
    'max_size': (1200, 800)      # 最大尺寸限制
})

ui.label('固定位置+自定义图标窗口')
ui.run(native=True)
```

4. 全屏启动 + 动态调整窗口

```
from nicegui import app, ui

# 初始全屏启动
app.native.window_args['fullscreen'] = True

# 动态切换全屏/窗口模式
def toggle_fullscreen():
    current_state = app.native.window.fullscreen
    app.native.window.fullscreen = not current_state

ui.button('切换全屏', on_click=toggle_fullscreen)
ui.run(native=True)
```

- `app.native.window` 是 `PyWebView` 的窗口实例，启动后可通过该对象动态修改窗口属性（如 `fullscreen`、`size`、`position`）。

四、关键注意事项

1. 生效时机

`app.native.window_args` 的修改必须在 `ui.run(native=True)` 之前完成，启动后修改该字典不会生效；若需运行时调整窗口属性，需直接操作 `app.native.window`（`PyWebView` 实例）。

2. 跨平台兼容性

- Windows：支持所有核心配置项，透明 / 无边框效果最佳；
- macOS：`transparent` 仅在 macOS 10.14 + 生效，`frameless` 窗口的拖动需自定义实现；
- Linux：依赖系统桌面环境（如 GNOME/KDE），部分配置项（如 `transparent`）可能无效。

3. 依赖安装

桌面模式依赖 PyWebView 的底层库，需额外安装对应系统的依赖：

```
# windows (默认已包含)
pip install nicegui[pywebview]

# macOS
brew install pyobjc-framework-webkit
pip install nicegui[pywebview]

# Linux (Debian/Ubuntu)
sudo apt install python3-gi python3-gi-cairo gir1.2-webkit2-4.0
pip install nicegui[pywebview]
```

4. 与 Web 模式的冲突

- 若 `ui.run()` 未设置 `native=True`，`app.native.window_args` 和 `app.native.window` 均为 `None`，操作会报错；
- 建议通过 `if app.native.window:` 判断是否为桌面模式后再操作：

```
if app.native.window:
    app.native.window.always_on_top = True # 仅桌面模式执行
```

5. 性能与资源占用

- 透明 / 无边框窗口会增加少量性能开销，低配置设备可能出现卡顿；
- 避免同时启用 `transparent` 和 `fullscreen`，部分系统会出现渲染异常。

NiceGUI 中 `app.native.start_args` 的详细解析

`app.native.start_args` 是 NiceGUI 针对 **PyWebView 桌面窗口启动流程** 暴露的配置属性，用于定制化 PyWebView `start()` 函数的入参，是桌面模式下控制窗口运行时行为（如事件循环、多线程、调试模式）的核心入口。它与 `app.native.window_args`（窗口外观 / 尺寸配置）互补，前者聚焦「启动逻辑」，后者聚焦「窗口属性」，仅在 `ui.run(native=True)`（桌面模式）下生效。

一、核心定位与底层原理

1. 适用场景

NiceGUI 的桌面模式基于 PyWebView 实现：

- `app.native.window_args` → 传递给 PyWebView `create_window()`（创建窗口时的静态属性）；
 - `app.native.start_args` → 传递给 PyWebView `start()`（启动窗口事件循环时的运行时参数）。
- 两者共同决定桌面窗口的最终行为，`start_args` 主要控制「窗口如何运行」（而非「窗口长什么样」）。

2. 底层依赖

PyWebView 的 `start()` 函数负责启动窗口的事件循环、绑定系统原生消息处理，`app.native.start_args` 本质是该函数的参数字典，NiceGUI 将其封装为 `app.native` 的属性，开发者可通过修改该字典覆盖默认启动行为。

二、核心配置项详解

`app.native.start_args` 是一个字典，支持 PyWebView `start()` 函数的所有参数（不同操作系统 / PyWebView 版本支持度略有差异），核心配置项如下：

配置项	类型	默认值 (NiceGUI)	作用	跨平台支持
<code>debug</code>	bool	<code>False</code>	启用 PyWebView 调试模式：- 输出详细日志（含前端 / 后端通信）；- 前端可打开开发者工具（F12）；- 便于排查桌面窗口渲染 / 通信问题	全平台
<code>http_server</code>	bool	<code>True</code>	是否启动内置 HTTP 服务器：- <code>True</code> ：NiceGUI 通过本地 HTTP 服务加载前端资源（与 Web 模式一致）；- <code>False</code> ：以本地文件模式加载（ <code>file://</code> 协议），需处理跨域 / 资源路径问题	全平台
<code>gui</code>	str	<code>None</code> （自动选择）	指定 PyWebView 使用的底层 GUI 引擎：- <code>'qt'</code> ：Qt WebEngine（跨平台，兼容性好）；- <code>'gtk'</code> ：GTK WebKit（Linux）；- <code>'webkit'</code> ：macOS 原生 WebKit；- <code>'edgechromium'</code> ：Windows Edge 内核	按系统区分
<code>loop</code>	<code>asyncio.AbstractEventLoop</code>	<code>None</code>	指定自定义的 <code>asyncio</code> 事件循环：- 用于集成第三方异步框架（如 Trio、Curio）；- 默认为 NiceGUI 创建的默认事件循环	全平台
<code>private_mode</code>	bool	<code>False</code>	启用私有模式：- 禁用缓存、Cookie、本地存储；- 适合隐私敏感的桌面应用	Windows/macOS

扩展配置项（PyWebView 高级参数）

配置项	适用系统	作用
<code>backend</code>	全平台	别名 <code>gui</code> ，优先级低于 <code>gui</code> （兼容旧版 PyWebView）
<code>ssl</code>	全平台	为内置 HTTP 服务器启用 SSL（需传入证书路径）
<code>service_workers</code>	bool	启用 / 禁用 Service Worker（前端离线缓存）

三、使用方式与示例

1. 基础用法：启用调试模式 + 指定 GUI 引擎

```
from nicegui import app, ui

# 配置窗口启动参数（启动逻辑）
app.native.start_args.update({
    'debug': True, # 启用调试模式（F12打开开发者工具）
    'gui': 'qt' # 强制使用Qt WebEngine引擎（跨平台兼容）
})

# 配置窗口属性（外观）
app.native.window_args.update({
    'title': '调试模式示例',
    'width': 900,
    'height': 600
})

ui.label('桌面模式调试示例（按F12打开开发者工具）')
ui.run(native=True)
```

- 调试模式下，控制台会输出 PyWebView 的详细日志（如 WebSocket 连接、资源加载）；
- 前端可通过 F12 调试组件、查看网络请求，与浏览器调试体验一致。

2. 进阶示例：禁用 HTTP 服务器（本地文件模式）

```
from nicegui import app, ui
import os

# 配置启动参数：禁用内置HTTP服务器，以本地文件模式运行
app.native.start_args['http_server'] = False
# 需手动指定静态资源路径（本地文件模式下NiceGUI无法自动映射）
app.add_static_files('/static', os.path.join(os.getcwd(), 'static'))

app.native.window_args.update({
    'title': '本地文件模式示例',
    'width': 800,
    'height': 500
})

ui.label('本地文件模式运行（无HTTP服务器）')
# 注意：本地文件模式下部分功能（如WebSocket）可能受限
ui.run(native=True)
```

- 适用场景：离线运行的桌面应用（无需启动 HTTP 服务）；
- 限制：WebSocket、跨域请求可能失效，需额外处理资源路径。

3. 自定义事件循环（集成异步框架）

```
from nicegui import app, ui
import asyncio

# 创建自定义asyncio事件循环
custom_loop = asyncio.new_event_loop()
asyncio.set_event_loop(custom_loop)

# 配置启动参数：使用自定义事件循环
app.native.start_args['loop'] = custom_loop

# 后台异步任务（基于自定义循环）
async def background_task():
    while True:
        print('自定义循环的后台任务')
        await asyncio.sleep(2)

# 启动任务
custom_loop.create_task(background_task())

ui.label('自定义事件循环示例')
ui.run(native=True)
```

- 适用场景：需集成第三方异步框架（如 Trio）或精细控制事件循环的场景；
- 注意：自定义循环需确保线程安全，避免与 NiceGUI 内置循环冲突。

4. 启用私有模式 + SSL（安全场景）

```
from nicegui import app, ui

# 配置启动参数：私有模式+SSL
app.native.start_args.update({
    'private_mode': True, # 禁用缓存/Cookie
    'ssl': ('cert.pem', 'key.pem') # 传入SSL证书（需自行生成）
})

app.native.window_args['title'] = '安全模式示例'
ui.label('私有模式+SSL加密运行')
ui.run(native=True, host='0.0.0.0', port=8443) # SSL需指定端口
```

- 适用场景：处理敏感数据的桌面应用（如密码管理、金融工具）；
 - 注意：SSL 证书需自行生成（如通过 OpenSSL），且仅在 http_server=True 时生效。
-

四、关键注意事项

1. 生效时机

- `app.native.start_args` 的修改必须在 `ui.run(native=True)` 之前完成，启动后修改无效；
- 若需运行时调整事件循环，需直接操作 `app.native.start_args['loop']` 对应的循环对象。

2. 与 `window_args` 的分工

维度	<code>app.native.window_args</code>	<code>app.native.start_args</code>
作用	窗口静态属性（尺寸、标题、边框）	窗口运行时行为（调试、事件循环）
底层函数	<code>PyWebView create_window()</code>	<code>PyWebView start()</code>
生效阶段	窗口创建时	窗口启动（事件循环）时

3. 跨平台兼容性

- `gui` 参数：Windows 优先 `edgechromium`，macOS 优先 `webkit`，Linux 优先 `gtk`；强制指定 `qt` 可获得跨平台一致体验，但需安装 `PyQt5/PySide2`；
- `private_mode`：Linux 桌面环境（如 GNOME）可能不支持，需测试验证；
- `ssl`：本地文件模式（`http_server=False`）下无效。

4. 依赖与版本问题

- PyWebView 版本差异：`start()` 参数在 PyWebView 4.x 与 3.x 中略有不同（如 `backend` → `gui`），NiceGUI 默认适配最新版；
- 调试模式限制：macOS 下 `debug=True` 可能导致窗口卡顿，建议仅开发环境启用。

5. 错误处理

- 若 `start_args` 配置错误（如指定不存在的 `gui` 引擎），PyWebView 会抛出 `NotImplementedError`，需捕获并调整配置：

```
try:
    ui.run(native=True)
except NotImplementedError as e:
    print(f'启动失败: {e}')
    app.native.start_args['gui'] = None # 恢复自动选择
    ui.run(native=True)
```

NiceGUI 中 `app.native.settings` 的详细解析

`app.native.settings` 是 NiceGUI 在桌面模式（`ui.run(native=True)`）下暴露的 PyWebView 全局设置入口，用于配置 PyWebView 底层运行时的全局行为（如资源加载策略、JS 交互权限、缓存规则等），是比 `app.native.window_args`（窗口属性）、`app.native.start_args`（启动参数）更底层的全局配置项。它面向 PyWebView 的核心运行时规则，而非单个窗口或启动流程，仅在桌面模式下有效。

一、核心定位与底层原理

1. 适用场景

NiceGUI 桌面模式基于 PyWebView 实现，PyWebView 提供 `webview.settings` 全局配置对象，用于统一管控所有窗口的底层行为（如是否允许 JS 调用 Python、是否启用缓存、资源加载限制等）。`app.native.settings` 正是对该全局对象的直接封装，开发者可通过修改该属性实现跨窗口的全局规则配置。

2. 核心价值

- 区别于 `window_args`（单窗口属性）和 `start_args`（启动流程），`settings` 聚焦 **全局运行时规则**；
- 所有通过 PyWebView 创建的窗口（包括 NiceGUI 的主窗口）都会继承 `settings` 的配置；
- 支持运行时动态修改（部分配置需重启窗口生效）。

二、核心配置项详解

`app.native.settings` 是一个类字典对象（PyWebView 的 `Settings` 实例），支持以下核心配置项（跨平台通用，部分项受系统限制）：

配置项	类型	默认值	作用	关键说明
<code>ALLOW_FILE_ACCESS_FROM_FILE_URLS</code>	bool	<code>False</code>	允许本地文件 URL（ <code>file:///</code> ）访问其他本地文件	本地文件模式下加载静态资源需开启，存在安全风险
<code>ALLOW_UNIVERSAL_ACCESS_FROM_FILE_URLS</code>	bool	<code>False</code>	允许本地文件 URL 访问任意资源（包括网络）	比上一项权限更大，仅开发环境启用
<code>CACHE_ENABLED</code>	bool	<code>True</code>	启用 HTTP / 本地文件缓存	禁用后可强制每次加载最新资源，适合调试
<code>DEVELOPER_TOOLS_ENABLED</code>	bool	<code>False</code>	启用开发者工具（F12）	等价于 <code>start_args['debug']</code> ，优先级更高
<code>JAVASCRIPT_ENABLED</code>	bool	<code>True</code>	启用 JavaScript 执行	禁用后 NiceGUI 前端交互会失效（核心依赖 JS）
<code>JAVASCRIPT_CAN_OPEN_WINDOW</code>	bool	<code>False</code>	允许 JS 通过 <code>window.open()</code> 创建新窗口	NiceGUI 的 <code>ui.navigate</code> 不依赖此配置
<code>JAVASCRIPT_CAN_ACCESS_CLIPBOARD</code>	bool	<code>False</code>	允许 JS 读写剪贴板	启用后前端可通过 <code>navigator.clipboard</code> 操作剪贴板
<code>LOAD_IMAGES</code>	bool	<code>True</code>	加载图片资源	禁用后可提升低性能设备运行速度
<code>LOCAL_STORAGE_ENABLED</code>	bool	<code>True</code>	启用前端 LocalStorage	NiceGUI 的 <code>app.storage.local</code> 依赖此配置
<code>REMOTE_DEBUGGING_PORT</code>	int	<code>None</code>	远程调试端口（如 9222）	可通过 Chrome DevTools 远程调试窗口
<code>USER_AGENT</code>	str	系统默认	自定义浏览器 User-Agent	用于模拟不同浏览器 / 设备的请求

三、使用方式与示例

1. 基础用法：启用开发者工具 + 禁用缓存

```
from nicegui import app, ui

# 仅桌面模式下配置settings
if app.native:
    # 启用开发者工具（F12）
    app.native.settings.DEVELOPER_TOOLS_ENABLED = True
    # 禁用缓存（强制加载最新资源）
    app.native.settings.CACHE_ENABLED = False
    # 允许JS访问剪贴板
    app.native.settings.JAVASCRIPT_CAN_ACCESS_CLIPBOARD = True

# 配置窗口属性
app.native.window_args.update({
    'title': '全局设置示例',
    'width': 900,
    'height': 600
})

# 测试剪贴板功能
def copy_to_clipboard():
    ui.run_javascript('navigator.clipboard.writeText("测试剪贴板内容")')

ui.button('复制文本到剪贴板', on_click=copy_to_clipboard)
ui.run(native=True)
```

- 效果：启动后按 F12 可打开开发者工具，修改前端代码无需清理缓存，点击按钮可直接写入剪贴板。

2. 进阶示例：本地文件模式下允许资源访问

```
from nicegui import app, ui
import os

# 桌面模式配置
if app.native:
    # 禁用HTTP服务器（本地文件模式）
    app.native.start_args['http_server'] = False
    # 允许本地文件URL访问其他文件（关键配置）
    app.native.settings.ALLOW_FILE_ACCESS_FROM_FILE_URLS = True
    # 允许本地文件URL访问网络资源（可选）
    app.native.settings.ALLOW_UNIVERSAL_ACCESS_FROM_FILE_URLS = True

# 挂载静态资源（本地文件模式需手动指定）
static_dir = os.path.join(os.getcwd(), 'static')
app.add_static_files('/static', static_dir)

# 加载本地图片（依赖ALLOW_FILE_ACCESS_FROM_FILE_URLS=True）
ui.image('/static/test.png').classes('w-64')
```

```
ui.run(native=True)
```

- 核心：本地文件模式（`http_server=False`）下，默认禁止本地文件访问其他资源，需开启 `ALLOW_FILE_ACCESS_FROM_FILE_URLS` 才能加载本地图片 / JS/CSS。

3. 自定义 User-Agent + 启用远程调试

```
from nicegui import app, ui

if app.native:
    # 自定义User-Agent（模拟Chrome浏览器）
    app.native.settings.USER_AGENT = 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36'
    # 启用远程调试（端口9222）
    app.native.settings.REMOTE_DEBUGGING_PORT = 9222
    # 禁用LocalStorage（隐私保护）
    app.native.settings.LOCAL_STORAGE_ENABLED = False

app.native.window_args['title'] = '远程调试+自定义UA示例'
ui.label('访问 chrome://inspect 可远程调试该窗口')
ui.run(native=True)
```

- 远程调试：启动后打开 Chrome 浏览器，访问 `chrome://inspect`，可识别到该桌面窗口并进行调试；
- 自定义 UA：适用于需要模拟特定浏览器的场景（如兼容特定网页）。

4. 运行时动态修改设置

```
from nicegui import app, ui

# 初始禁用图片加载
if app.native:
    app.native.settings.LOAD_IMAGES = False

# 动态切换图片加载状态
def toggle_image_load():
    if app.native:
        app.native.settings.LOAD_IMAGES = not app.native.settings.LOAD_IMAGES
        # 刷新页面使配置生效
        ui.run_javascript('window.location.reload()')

ui.button('切换图片加载状态', on_click=toggle_image_load)
ui.image('https://picsum.photos/200').classes('mt-4') # 测试图片加载
ui.run(native=True)
```

- 注意：部分设置（如 `LOAD_IMAGES`）修改后需刷新页面才能生效；
- 动态修改适合根据用户操作调整运行时规则（如低性能模式下禁用图片加载）。

四、关键注意事项

1. 生效范围与优先级

- `app.native.settings` 是**全局配置**，对所有 PyWebView 窗口生效（包括 NiceGUI 主窗口、通过 `webview.create_window` 创建的子窗口）；
- 优先级：`settings` 全局配置 > `start_args` 启动参数 > `window_args` 窗口属性；例如 `settings.DEVELOPER_TOOLS_ENABLED=True` 会覆盖 `start_args['debug']=False`。

2. 安全风险提示

- `ALLOW_UNIVERSAL_ACCESS_FROM_FILE_URLS=True`：允许本地文件访问网络资源，可能被恶意 JS 利用，生产环境禁止启用；
- `JAVASCRIPT_CAN_ACCESS_CLIPBOARD=True`：需确保前端代码可信，避免剪贴板数据泄露；
- 建议：开发环境可放宽配置，生产环境仅启用必要权限。

3. 跨平台兼容性

- Windows/macOS：支持所有核心配置项，`REMOTE_DEBUGGING_PORT`、`USER_AGENT` 效果稳定；
- Linux：`ALLOW_FILE_ACCESS_FROM_FILE_URLS` 可能受桌面环境限制（如 GNOME 需额外配置），`REMOTE_DEBUGGING_PORT` 仅部分浏览器支持；
- 禁用 `JAVASCRIPT_ENABLED` 会导致 NiceGUI 完全失效（NiceGUI 前端基于 Vue/JS）。

4. 生效时机

- 大部分配置（如 `CACHE_ENABLED`、`LOAD_IMAGES`）修改后需刷新页面（`window.location.reload()`）才能生效；
- 少数配置（如 `REMOTE_DEBUGGING_PORT`）需重启应用才能生效；
- 建议：核心配置在 `ui.run()` 前完成，运行时动态修改仅用于临时调整。

5. 与 Web 模式的区别

- Web 模式下 `app.native` 为 `None`，`app.native.settings` 不存在，操作会报错；
- 建议通过 `if app.native:` 判断模式后再配置：

```
if app.native:
    app.native.settings.DEVELOPER_TOOLS_ENABLED = True
else:
    print('当前为web模式，settings配置无效')
```

NiceGUI 中 app.native.main_window 对象的详细解析

`app.native.main_window` 是 NiceGUI 桌面模式 (`ui.run(native=True)`) 下暴露的**主窗口实例**，本质是 PyWebView 库的 `window` 对象，是操作桌面窗口运行时行为的核心入口。它区别于静态配置 (`window_args` / `start_args` / `settings`)，聚焦「运行时动态控制」—— 可在应用启动后实时调整窗口状态（如尺寸、位置、全屏）、触发窗口行为（如最小化、关闭）、交互前端页面（如执行 JS、获取 DOM），是桌面应用定制化的最终落地载体。

一、核心定位与底层原理

1. 适用场景

- 仅在 `ui.run(native=True)` 启动的桌面模式下存在，Web 模式下 `app.native.main_window` 为 `None`；
- 是 PyWebView `window` 类的实例，继承了 PyWebView 所有窗口操作 API，NiceGUI 仅做透传封装，无额外限制。

2. 与其他 native 属性的关系

组件	作用	与 main_window 的关系
<code>window_args</code>	窗口创建时的静态属性配置	初始化 <code>main_window</code> 的基础属性
<code>start_args</code>	窗口启动时的运行时参数	决定 <code>main_window</code> 的事件循环 / 引擎配置
<code>settings</code>	全局运行时规则	约束 <code>main_window</code> 的底层行为（如 JS 权限）
<code>main_window</code>	运行时窗口实例	可动态覆盖上述配置的最终操作对象

二、核心属性与方法详解

`app.native.main_window` 继承 PyWebView `window` 对象的所有属性和方法，核心可分为「状态查询 / 修改」「窗口行为控制」「前后端交互」「事件监听」四大类，以下是高频使用的核心能力：

（一）状态查询 / 修改（实时获取 / 调整窗口属性）

属性 / 方法	类型 / 参数	作用
<code>width</code> / <code>height</code>	int（可读可写）	获取 / 设置窗口宽度 / 高度（像素），修改后立即生效
<code>x</code> / <code>y</code>	int（可读可写）	获取 / 设置窗口左上角坐标（相对于屏幕），修改后窗口立即移动
<code>title</code>	str（可读可写）	获取 / 修改窗口标题栏文本
<code>fullscreen</code>	bool（可读可写）	获取 / 设置窗口是否全屏（True = 全屏，False = 窗口模式）

属性 / 方法	类型 / 参数	作用
<code>min_size / max_size</code>	tuple (可读可写)	获取 / 设置窗口最小 / 最大尺寸 (如 (400, 300))
<code>resizable</code>	bool (可读可写)	获取 / 设置窗口是否可缩放
<code>always_on_top</code>	bool (可读可写)	获取 / 设置窗口是否置顶 (总在其他窗口上层)
<code>visible</code>	bool (可读可写)	获取 / 设置窗口是否可见 (False = 隐藏, True = 显示)
<code>transparent</code>	bool (只读)	查询窗口是否透明 (需初始化时通过 <code>window_args</code> 设置, 运行时不可改)
<code>frameless</code>	bool (只读)	查询窗口是否无边框 (需初始化时设置, 运行时不可改)

示例：实时调整窗口尺寸与位置

```
from nicegui import app, ui

# 初始化窗口属性
app.native.window_args.update({
    'title': '动态调整窗口示例',
    'width': 800,
    'height': 600,
    'x': 100,
    'y': 100
})

# 放大窗口
def enlarge_window():
    if app.native.main_window:
        app.native.main_window.width += 100
        app.native.main_window.height += 50

# 移动窗口到屏幕中央
def center_window():
    if app.native.main_window:
        # 获取屏幕尺寸 (需通过JS获取)
        ui.run_javascript('''
            return {
                screen_width: window.screen.width,
                screen_height: window.screen.height
            }
        ''', callback=lambda data: adjust_position(data))

def adjust_position(data):
    w = app.native.main_window.width
    h = app.native.main_window.height
```

```
app.native.main_window.x = (data['screen_width'] - w) // 2
app.native.main_window.y = (data['screen_height'] - h) // 2
```

```
ui.button('放大窗口', on_click=enlarge_window)
ui.button('居中窗口', on_click=center_window)
ui.run(native=True)
```

(二) 窗口行为控制（触发窗口动作）

方法	参数	作用
<code>minimize()</code>	无	最小化窗口
<code>maximize()</code>	无	最大化窗口
<code>restore()</code>	无	还原窗口（从最大化 / 最小化恢复）
<code>close()</code>	无	关闭窗口（等价于点击标题栏关闭按钮，会触发 <code>on_closed</code> 事件）
<code>destroy()</code>	无	强制销毁窗口（不触发关闭事件，慎用）
<code>set_icon(icon_path)</code>	icon_path: str	动态修改窗口图标（支持 .ico/.png，跨平台）
<code>toggle_fullscreen()</code>	无	切换全屏 / 窗口模式（等价于修改 <code>fullscreen</code> 属性）

示例：窗口行为控制组合

```
from nicegui import app, ui

def control_window(action):
    w = app.native.main_window
    if not w:
        return
    if action == 'min':
        w.minimize()
    elif action == 'max':
        w.maximize()
    elif action == 'restore':
        w.restore()
    elif action == 'close':
        w.close()
    elif action == 'icon':
        w.set_icon('./custom_icon.ico') # 动态修改图标

ui.button('最小化', on_click=lambda: control_window('min'))
ui.button('最大化', on_click=lambda: control_window('max'))
ui.button('还原', on_click=lambda: control_window('restore'))
ui.button('修改图标', on_click=lambda: control_window('icon'))
ui.button('关闭', on_click=lambda: control_window('close'))
ui.run(native=True, title='窗口行为控制示例')
```

(三) 前后端交互（窗口级页面操作）

方法	参数	作用
<code>evaluate_js(js_code)</code>	js_code: str	同步执行前端 JS 代码，返回执行结果（如获取 DOM 值、调用前端函数）
<code>async evaluate_js_async(js)</code>	js: str	异步执行 JS 代码（适合耗时操作，如请求数据）
<code>load_url(url)</code>	url: str	让窗口加载指定 URL（可替换当前 NiceGUI 页面，如加载外部网页）
<code>load_html(html)</code>	html: str	让窗口加载自定义 HTML 内容（替代当前页面）
<code>get_current_url()</code>	无	获取窗口当前加载的 URL

示例：执行 JS 获取 DOM 值 + 加载外部网页

```
from nicegui import app, ui

# 执行JS获取输入框值
def get_input_value():
    if app.native.main_window:
        # 同步执行JS，获取id为"input1"的输入框值
        value =
app.native.main_window.evaluate_js('document.getElementById("input1").value')
    ui.notify(f'输入框值: {value}')

# 加载外部网页
def load_baidu():
    if app.native.main_window:
        app.native.main_window.load_url('https://www.baidu.com')

ui.input('测试输入框').id('input1')
ui.button('获取输入框值', on_click=get_input_value)
ui.button('加载百度', on_click=load_baidu)
ui.run(native=True)
```

(四) 事件监听（监听窗口生命周期事件）

`app.native.main_window` 支持监听窗口的原生事件，通过 `events` 属性绑定回调函数，核心事件如下：

事件名	触发时机	回调参数
<code>on_closed</code>	窗口关闭时	无
<code>on_closing</code>	窗口即将关闭时（可阻止关闭）	无
<code>on_resized</code>	窗口尺寸变化时	(width, height)

事件名	触发时机	回调参数
<code>on_moved</code>	窗口位置变化时	(x, y)
<code>on_minimized</code>	窗口最小化时	无
<code>on_maximized</code>	窗口最大化时	无
<code>on_restored</code>	窗口还原时	无

示例：监听窗口关闭 / 尺寸变化事件

```
from nicegui import app, ui

def setup_window_events():
    w = app.native.main_window
    if not w:
        return

    # 窗口即将关闭时提示
    def on_closing():
        ui.notify('窗口即将关闭! ')
        # 可通过返回False阻止关闭（部分系统支持）
        # return False

    # 窗口尺寸变化时打印新尺寸
    def on_resized(width, height):
        print(f'窗口尺寸变为: {width}x{height}')

    # 绑定事件
    w.events.on_closing += on_closing
    w.events.on_resized += on_resized
    # 解绑事件（如需）: w.events.on_resized -= on_resized

# 启动后绑定事件
ui.run(native=True, on_startup=setup_window_events)
```

三、高级使用场景

1. 无边框窗口自定义拖动

无边框窗口（`frameless=True`）默认无法拖动，需通过 `main_window` 结合 JS 实现自定义拖动区域：

```
from nicegui import app, ui

# 配置无边框窗口
app.native.window_args['frameless'] = True

# 自定义拖动区域的JS
drag_js = ''
```

```

// 给id为"drag-area"的元素绑定拖动事件
let dragArea = document.getElementById('drag-area');
let isDragging = false;
let startX, startY, windowX, windowY;

dragArea.addEventListener('mousedown', (e) => {
    isDragging = true;
    startX = e.clientX;
    startY = e.clientY;
    // 获取当前窗口位置（通过Python调用返回）
    windowX = await window.pywebview.api.get_window_x();
    windowY = await window.pywebview.api.get_window_y();
});

document.addEventListener('mousemove', (e) => {
    if (!isDragging) return;
    const dx = e.clientX - startX;
    const dy = e.clientY - startY;
    // 调用Python方法移动窗口
    window.pywebview.api.move_window(windowX + dx, windowY + dy);
});

document.addEventListener('mouseup', () => {
    isDragging = false;
});
...

# 暴露Python方法给JS调用
def get_window_x():
    return app.native.main_window.x

def get_window_y():
    return app.native.main_window.y

def move_window(x, y):
    app.native.main_window.x = x
    app.native.main_window.y = y

ui.expose_api('get_window_x', get_window_x)
ui.expose_api('get_window_y', get_window_y)
ui.expose_api('move_window', move_window)

# 自定义拖动区域
ui.row().id('drag-area').classes('w-full h-10 bg-gray-800').content('拖动此区域移动窗口')
ui.run_javascript(drag_js)
ui.run(native=True)

```

2. 多窗口管理（主窗口 + 子窗口）

`app.native.main_window` 是主窗口，可通过 `PyWebView` 原生 API 创建子窗口，并通过主窗口协调管理：

```
from nicegui import app, ui
import webview # 直接使用PyWebView库

def open_sub_window():
    # 创建子窗口
    sub_window = webview.create_window(
        title='子窗口',
        url='https://nicegui.io',
        width=600,
        height=400
    )
    # 主窗口居中，子窗口定位在主窗口右侧
    main_x = app.native.main_window.x
    main_y = app.native.main_window.y
    sub_window.x = main_x + app.native.main_window.width + 20
    sub_window.y = main_y

ui.button('打开子窗口', on_click=open_sub_window)
ui.run(native=True, title='主窗口')
```

3. 窗口透明 + 动态调整透明度

(仅 Windows/macOS 支持，需初始化时设置 `transparent=True`)

```
from nicegui import app, ui

# 初始化透明无边框窗口
app.native.window_args.update({
    'frameless': True,
    'transparent': True,
    'width': 500,
    'height': 300
})

# windows下通过API调整透明度（需调用系统原生API）
import ctypes
def set_opacity(opacity):
    if app.native.main_window and os.name == 'nt':
        # 获取窗口句柄
        hwnd = app.native.main_window.get_handle()
        # 设置透明度（0-255，0=全透，255=不透明）
        ctypes.windll.user32.SetLayeredWindowAttributes(hwnd, 0, int(opacity), 2)

ui.slider(min=0, max=255, step=1, value=200).on_change(lambda e: set_opacity(e.value))
ui.run(native=True)
```

四、关键注意事项

1. 访问时机

- `app.native.main_window` 仅在 `ui.run(native=True)` 启动后才初始化完成，**启动前访问会报错**；
- 若需在启动后立即操作，需通过 `on_startup` 回调：

```
def on_app_start():  
    # 启动后立即置顶窗口  
    app.native.main_window.always_on_top = True  
  
ui.run(native=True, on_startup=on_app_start)
```

2. 跨平台兼容性

功能	Windows	macOS	Linux	备注
尺寸 / 位置修改	✓	✓	✓	全平台支持
置顶 / 最小化 / 最大化	✓	✓	✓	Linux 需桌面环境支持
透明窗口 + 透明度调整	✓	✓	✗	Linux 无统一透明 API
自定义图标	✓	✓	✓	Linux 需.svg 格式
事件监听 (<code>on_resized</code>)	✓	✓	部分支持	KDE/GNOME 差异

3. 线程安全

- `app.native.main_window` 的方法**非线程安全**，若在后台线程中操作，需通过 `asyncio` 或 `ui.call_from_thread` 包装：

```
def background_task():  
    # 后台线程中修改窗口标题  
    ui.call_from_thread(lambda: app.native.main_window.title = '新标题')  
  
ui.timer(5, background_task, once=True)
```

4. 与 NiceGUI 前端的冲突

- 调用 `load_url()` / `load_html()` 会替换当前 NiceGUI 页面，导致原有 UI 组件失效；
- 若需保留 NiceGUI 页面，建议通过 `ui.run_javascript()` 修改 DOM，而非直接加载新 URL。

5. 资源释放

- 调用 `close()` 会触发正常的窗口关闭流程（执行 `on_shutdown` 回调）；
- 调用 `destroy()` 会强制销毁窗口，可能导致资源泄漏（如数据库连接未关闭），仅应急使用。

